

Proposal for Orbital 26

Team Name:

[Please come up with a creative name for your team. Your team name should contain at least one English letter.]

- Procrastinot

Proposed Level of Achievement:

[Project Gemini or Apollo 11 or Artemis. Please read the descriptions of the levels of achievement on the Orbital website: <https://soc-n.us/orbital-loa> and choose only one.]

- Apollo 11

Motivation

[Please describe the motivation behind your project.]

- As university students ourselves, we have experienced firsthand the frustration of managing academic responsibilities scattered across multiple platforms. Assignment deadlines and exam dates arrive through the assignment or announcement tabs on Canvas, or through different platforms such as Coursemology, Code Crunch, and Outlook, as well as messages shared in WhatsApp or Telegram group chats. With no single place to see everything at once, it is easy to miss deadlines, underestimate upcoming workload, or leave large assignments untouched until it is too late. While some of us are more organised than others, all of us have felt the cognitive burden of manually piecing together a personal schedule from these disconnected sources. Existing tools such as calendar apps and to-do lists require entirely manual input, placing the organisational burden entirely on the student. We believe this problem is not a matter of student discipline, it is a systems problem, and one that software can meaningfully and conveniently solve, which can help to save students' time and effort.

Aim

[Please describe what you aim to achieve.]

- We hope to build an intelligent academic task management **web application** that consolidates information from multiple sources including platforms such as Canvas, Outlook, and even text inputs or manual entry into a single, unified dashboard. We aim to automatically extract deadlines and assignment details from these sources, help students visualise their upcoming workload weeks in advance, and break large assignments down into structured weekly milestones. Beyond individual use, we also aim to provide an administrative interface for system oversight and to give students full control over their own data. Ultimately, we hope to reduce the organisational overhead students face and help them approach their academic workload more proactively and with less stress.

User Stories

[Please describe what the users would be able to do with your system.]

1. As a student who wants to avoid manually re-entering deadlines from my course syllabi, I want to be able to upload a screenshot / text input and have the system automatically extract and populate my assignment deadlines.

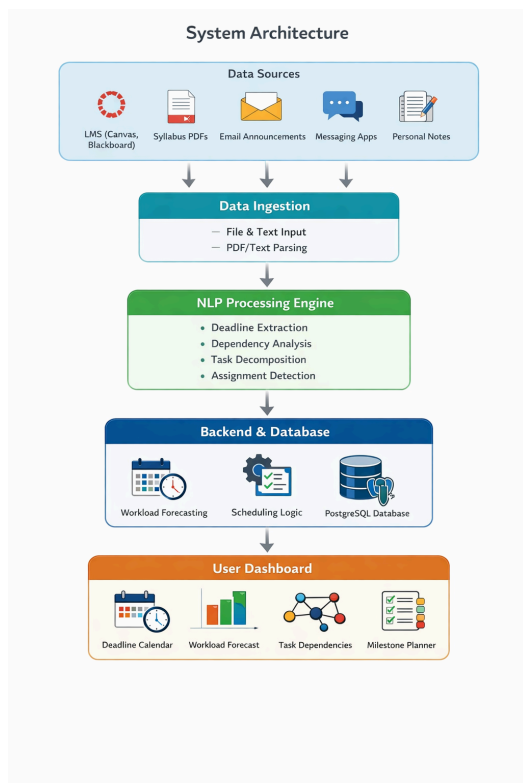
2. As a student preparing for exams, I want my exam and test dates to appear on my calendar alongside assignment deadlines so that I can see my full academic workload in one place.
3. As a student who wants to stay on top of a large assignment without leaving it to the last minute, I want to be able to select an assignment and have the system break it down into suggested weekly milestones with recommended completion dates.
4. As a student who uses Canvas to manage my courses, I want to be able to connect my Canvas account so that my assignments and due dates are automatically imported into the system without manual entry.
5. As a student who wants to plan my time effectively, I want to be able to see a visual forecast of my upcoming workload so that I can identify weeks where I have multiple deadlines and plan ahead accordingly.
6. As a student who works on group assignments, I want to be able to share a group related task list with my teammates so that everyone in the group can see the same milestones and deadlines.
7. As an administrator who needs to oversee the platform, I want to be able to view registered users, monitor system activity, and manage accounts so that I can ensure the platform is being used appropriately and resolve issues when they arise.
8. As an administrator who wants to understand how the platform is being used, I want to be able to view aggregate usage statistics — such as the number of syllabi uploaded and active users per week — so that I can identify whether the system is delivering value.

Features

[Please list down the key features of your system with a rough timeline for completion.]

1. Feature 1 (core): Syllabus PDF & Text Extraction
 - Upload PDF syllabus or paste text
 - Use Claude API to automatically extract assignment names, due dates, exam dates and deliverables
 - Parse vague dates ("mid-October") into structured formats
 - Store extracted assignments in database
2. Feature 2 (core): Canvas LMS Integration
 - Connect Canvas account via API token
 - Automatically sync assignments, due dates, and course info
 - Real-time sync option or manual refresh
 - Handle multiple Canvas courses
3. Feature 3 (core): Unified Assignment Dashboard
 - Single view of all assignments, tests and exams from all sources (PDF, Canvas, manual)
 - Filter by course, date range, deliverable type
 - Search and sort assignments
 - Mark assignments as completed/in-progress
4. Feature 4 (core): Workload Forecasting & Visualization
 - Calendar view showing deadlines, tests, exam dates with workload per week
 - "Heat map" showing high-stress weeks (multiple deadlines)
 - Estimated hours per assignment (extracted from syllabus + user input)
 - Weekly workload summary (total hours, number of assignments)
5. Feature 5 (core): Assignment Decomposition
 - Select an assignment → system suggests weekly milestones
 - Use Claude to break down large assignments into subtasks
 - Set recommended completion dates for each milestone
 - Students can customize suggested milestones
6. Feature 6 (extension): Collaborative Team Features
 - Create/join study groups or project teams
 - Share assignment lists and milestones with teammates
 - View teammates' progress on shared assignments
 - Team-level workload alerts (notify when everyone is overloaded)

7. Feature 7 (extension): Admin Dashboard
 - View registered users, account creation date, last login
 - Monitor system activity: total syllabi uploaded, active users per week
 - View platform usage statistics (assignments extracted, milestones created)
 - User management: deactivate accounts, view user data
8. Feature 8 (extension): Data Export & Privacy
 - Export personal assignments as CSV, iCal, or PDF report
 - Download all personal data as JSON (GDPR-style)
 - Delete account and all associated data
 - Data retention policy documentation
9. Feature 9 (extension): Mobile Responsive Design
 - Fully responsive application that works on phones and tablets
 - Mobile-optimized task list and workload view
 - Touch-friendly UI with mobile-optimized navigation and buttons
10. Feature 10 (extension): Comprehensive Testing Suite
 - Unit tests for extraction logic, date parsing, workload calculation (40+ tests)
 - Integration tests for API endpoints and database interactions (20+ tests)
 - End-to-end tests simulating real user workflows (Cypress, 5+ tests)
 - Test coverage target: 70%+



Timeline

[Please provide a rough timeline for completion in terms of Milestones.]

1. Milestone 1 - Technical Proof of Concept (by Early June, Week 2-3)

Goal: Demonstrate core technology integration works end-to-end

Feature 1 (Syllabus PDF Extraction): Basic version

- Can upload PDF, extract text, send to Claude API
- Returns raw JSON with assignment names and dates, due dates and exam dates
- Manual testing only, no validation

Feature 2 (Unified Dashboard): Minimal version

- Basic React frontend with hardcoded assignment data
- Can display assignments in a list
- No filters or interactivity yet

Feature 3 (Database): Schema designed and created

- PostgreSQL set up with Users, Courses, Assignments, Milestones tables
- FastAPI backend with basic CRUD endpoints
- Connected to frontend via API calls

Deliverables:

- Working PDF → assignment extraction pipeline
- Basic dashboard displaying mock data
- API endpoints functional
- GitHub repo with initial commits and basic README

2. Milestone 2 - Prototype with Core Features (by Mid-July, Week 6)

Goal: All core features functional and tested

Feature 1 (Syllabus PDF Extraction): Robust version

- Handles multiple PDF formats and edge cases
- Parses vague dates ("mid-October") correctly
- Validates extracted data before storing
- Unit tests with 20+ test cases

Feature 2 (Canvas LMS Integration): Complete

- Students can connect Canvas via API token
- Assignments automatically sync
- Handles course selection and filtering

Feature 3 (Unified Dashboard): Production-ready

- Display all assignments from all sources
- Filter by course, deadline date, status
- Search functionality
- Responsive design works on desktop & tablet
- Basic styling with clean UI

Feature 4 (Workload Forecasting): Core version

- Weekly calendar view showing deadlines
- Heat map highlighting high-stress weeks
- Estimated hours per assignment
- Visual bar chart of weekly workload

Feature 5 (Assignment Decomposition): Basic version

- Students select assignment → system suggests 4-6 weekly milestones
- Claude generates milestone names and dates
- Students can edit/customize milestones
- Milestones appear on main calendar

Deliverables:

- All core features working end-to-end
- User testing round 1 conducted (5-8 students)
- Unit + integration test suite (60+ tests, 65% coverage)
- Architecture Decision Records written
- Sprint retrospectives documented in wiki
- Basic user documentation

3. Milestone 3 - Extended System with Polish (by Late July/Early August, Week 10)

Goal: All features complete, tested, and production-ready

Feature 6 (Collaborative Team Features): Complete

- Create/join study groups
- Share assignment lists with specific teammates
- View teammates' milestones on shared assignments
- Team workload alerts

Feature 7 (Admin Dashboard): Complete

- Admin login and authentication
- View all users, activity logs, usage statistics
- Charts showing syllabi uploaded over time, active users
- Basic user management (view, deactivate accounts)

Feature 8 (Data Export & Privacy): Complete

- Export assignments as CSV, iCal, PDF
- Download full data as JSON
- Delete account functionality
- Privacy policy and documentation

Feature 9 (Mobile Responsive Design): Complete

- Full mobile responsiveness tested on iPhone/Android
- Mobile-optimized navigation and buttons
- Touch-friendly UI

Feature 10 (Comprehensive Testing): Complete

- End-to-end test suite (Cypress) with 8+ real user workflows
- Overall test coverage: 70%+
- Load testing: system handles 100+ concurrent users
- CI/CD pipeline with automated tests on every push

Security & Polish:

- Authentication & authorization properly implemented
- Input validation on all forms
- Error handling and user-friendly error messages
- Performance optimization (API response times < 200ms)
- Accessibility review (WCAG compliance check)

Documentation:

- Complete API documentation (OpenAPI/Swagger)
- Architecture documentation
- 5+ Architecture Decision Records (ADRs)

- User guide with screenshots
- Developer guide for future contributors
- Testing strategy and results document
- Sprint retrospectives and learnings

Deliverables:

- User testing round 2 conducted (5-8 students, different cohort)
- All features complete and working
- 70%+ test coverage with comprehensive test suite
- Production-ready codebase
- Complete documentation
- Demo video showing all features
- Presentation slides for Splashdown

Tech Stack

[Please list down the technologies that you are planning to use.]

1. Frontend:

- React.js (TypeScript optional, but recommended)
 - Reason: Component-based UI, good for dashboard
- TailwindCSS
 - Reason: Fast responsive styling
- Axios
 - Reason: Simple HTTP requests to backend

2. Backend:

- Python 3.11 + FastAPI
 - Reason: Modern, fast, type-safe, great for APIs
- PostgreSQL
 - Reason: Structured data storage (users, assignments, milestones)
- SQLAlchemy
 - Reason: Database ORM, type-safe queries

3. External APIs:

- Anthropic Claude API
 - Reason: Extract assignment details from PDFs and text
- Canvas API
 - Reason: Sync assignments directly from Canvas

4. Testing:

- pytest
 - Reason: Unit and integration tests for backend
- Cypress
 - Reason: End-to-end tests for frontend workflows

5. DevOps:

- GitHub
 - Reason: Version control, issue tracking, sprint planning
- GitHub Actions
 - Reason: Automated testing on every push

6. Deployment:

- Render or Railway (backend)
 - Reason: Free tier, simple deployment from GitHub
- Vercel (frontend, optional)
 - Reason: Optimized for React, auto-deploys

Qualifications

[Please elaborate on your technical skills and project experience of your team. This is required only for teams targeting Apollo 11 or Artemis.]

Our team consists of two first-year undergraduates, one studying Computer Science and one studying Business Analytics. Combined, we have a shared foundation in programming fundamentals, computational thinking, and algorithmic problem solving. We have hands-on experience with Python, Java, R, SQL and GitHub.

Beyond coursework, both members have participated in IDEATE 2025, where we developed experience working under time pressure, translating user needs into structured solutions, and presenting ideas clearly to an audience. Additionally, our Business Analytics member participated in the Business Analytics Case Competition, where they gained experience in problem structuring, data-driven decision making, and presenting actionable insights to a technical audience. This experience is particularly valuable for our project, as it has given us the ability to identify user pain points, translate those into concrete system requirements, and evaluate whether our features are genuinely solving the problems we set out to address.

Software Engineering

[Please showcase the software engineering knowledge of your team by providing specific details about what software engineering related techniques / practices you plan to apply in the development process of your project. This is required only for teams targeting Apollo 11 or Artemis.]

- Agile development process. We will organise our work into two-week sprints, each with a defined goal drawn from a maintained backlog. At the end of each sprint we will conduct a brief retrospective, documented in our repository wiki, reflecting on what was completed, what was blocked, and what needs to change. We will track all tasks and bugs using GitHub Issues, labelled by type and priority, so that our development process is transparent and auditable.
- Architecture decision records. For every significant technical decision — such as our choice of framework, database, authentication strategy, or how we handle PDF parsing — we will write a short Architecture Decision Record (ADR) stored in a /docs/adr/ folder. Each ADR captures the context, the options we considered, the decision we made, and the trade-offs accepted. This demonstrates that our design choices are deliberate and reasoned rather than arbitrary.
- Testing strategy. We will implement testing at three levels. Unit tests will cover individual functions and modules, particularly the deadline extraction logic, workload scoring, and task decomposition prompts. Integration tests will verify that our API endpoints correctly handle requests, interact with the database, and return the expected responses. End-to-end tests using Cypress will simulate real user journeys — for example, uploading a PDF syllabus and verifying that deadlines appear correctly on the dashboard.
- Usability testing. We will conduct two structured rounds of user testing with real students. The first round, held around the midpoint of development, will be observational — participants will be given realistic tasks and we will note where they hesitate, make errors, or express confusion. Findings will be logged as GitHub Issues and prioritised for the following sprint. The second round, held near the end of development, will assess whether those issues have been resolved and surface any

remaining friction. Both rounds will be documented with dates, participant counts, tasks administered, and a written summary of findings and the changes made in response.